

Goal

The goal of this project is to implement a BBR-like (Bottleneck Bandwidth and Round-trip propagation time) congestion control algorithm that regulates packet transmission using pacing and estimates of key network parameters such as BtlBw (bottleneck bandwidth) and RtProp (round-trip propagation time). The implementation is developed in multiple stages, with each stage progressively adding BBR mechanisms and evaluating the algorithm under different network and traffic conditions.

This implementation is based on the BBR congestion control algorithm described by Cardwell et al. in:

P9. [Cardwell17a] Neal Cardwell, Yuchung Cheng, C. Stephen Gunn, Soheil Hassas Yeganeh, and Van Jacobson. BBR: Congestion-based congestion control. *Communications of the ACM*, 60(2):58–66, February 2017. (<https://dl.acm.org/doi/10.1145/3009824>)

Core Concepts

Traditional congestion control algorithms such as TCP Reno rely on packet loss as the primary signal of congestion. However, this approach has several limitations:

- Congestion is detected only after packet loss occurs, which often means network buffers have already filled up, leading to increased latency.
- Modern networks often have large bandwidth-delay products (BDP) due to high-speed links and long-distance communication. When packet loss occurs, TCP reduces the congestion window (typically by half) and then slowly increases it again during congestion avoidance. This process takes time to refill the network pipe, resulting in underutilization of available bandwidth.

BBR addresses these issues by estimating the bottleneck bandwidth (BtlBw) and the minimum round-trip propagation time (RTprop), and setting the sending rate based on these values. By pacing packets and continuously updating these estimates, BBR keeps the network pipe full while avoiding large queues and unnecessary delays.

The basic network model used by BBR is illustrated in Figure 1. It shows how delivery rate and round-trip time change as the amount of inflight data increases, and highlights the optimal operating point at the bandwidth-delay product (BDP), where the link is fully utilized without building queues.

App-Limited Region

When the amount of data in flight is less than the **bandwidth-delay product (BDP)**, the sender is operating in the *app-limited region*. In this region, the network pipe is not fully utilized because the sender is not transmitting enough data to fill the link capacity. As a result, it is difficult to determine the true bottleneck bandwidth since the delivery rate depends on how much data the sender provides. However, the measured round-trip time (RTT) is usually close to the true propagation RTT (RT_{prop}) because queues have not yet formed in the network.

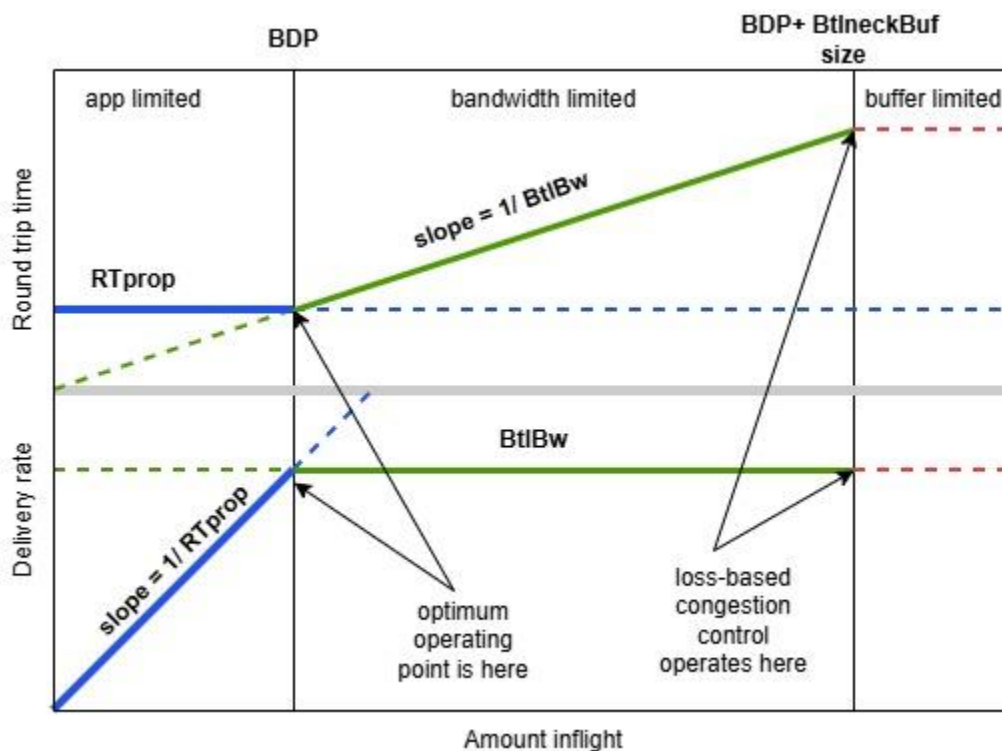


Figure 1: Relationship between delivery rate, round-trip time, and inflight data in the BBR model. The optimal operating point occurs at the bandwidth-delay product (BDP). *Adapted from Cardwell et al., 2017.*

Bandwidth-Limited Region

When the amount of inflight data approaches the BDP, the network pipe becomes fully utilized. In this region, the delivery rate stabilizes near the true bottleneck bandwidth ($BtlBw$), since the sender is now transmitting enough data to fully probe the link capacity. The RTT remains close to RT_{prop} because queues are still minimal. This region represents the optimal operating point, where the network is efficiently utilized without causing unnecessary queueing delays.

Buffer-Limited Region

If the sender continues to increase the amount of inflight data beyond the BDP, packets begin to accumulate in network buffers. In this region, the delivery rate cannot increase further because it is already limited by the bottleneck bandwidth. Instead, the additional packets create queueing delay, causing the measured RTT to increase. Traditional loss-based congestion control algorithms often operate in this region, where buffers fill up and packet loss may eventually occur.

BBR Estimation and Control

BBR estimates the true RTT (RT_{prop}) as the minimum observed RTT, because this occurs when the network queue is empty. The true bottleneck bandwidth ($BtlBw$) is estimated as the maximum observed delivery rate, since lower values may occur when the sender is not fully utilizing the link. Using these estimates, BBR calculates the bandwidth-delay product (BDP):

$$BDP = BtlBw \times RT_{prop}$$

BBR then controls the amount of inflight data using the congestion window:

$$cwnd = cwnd_gain \times BDP$$

Packets are transmitted using **pacing**, where the sender spaces packets evenly over time based on the estimated bandwidth:

$$pacing_rate = pacing_gain \times BtlBw$$

This approach allows BBR to keep the network pipe full while avoiding persistent queue buildup.

Implementation Details

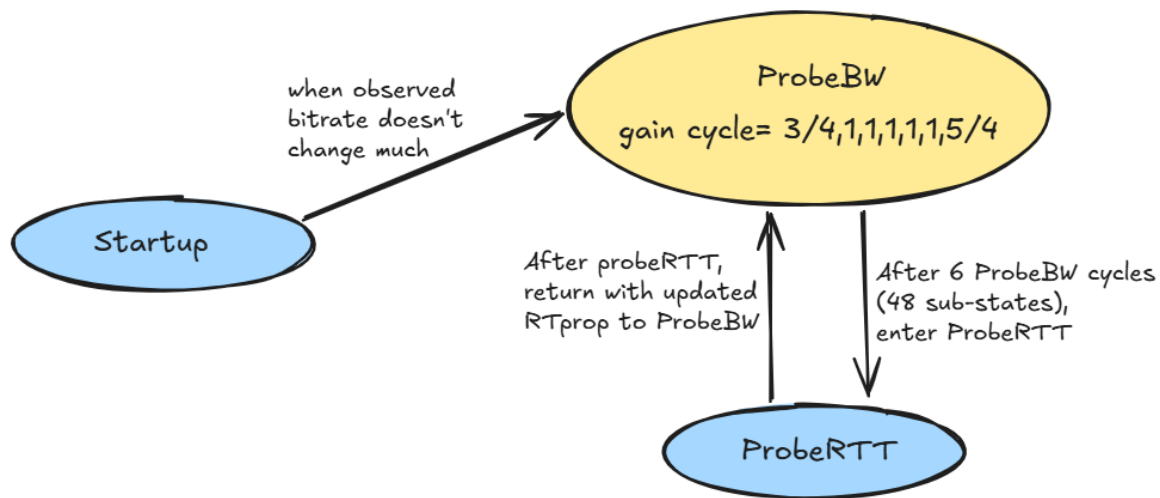


Figure 2: State transitions in the simplified BBR implementation used in this project. The algorithm begins in the Startup state to rapidly probe for available bandwidth, transitions to ProbeBW once the bitrate stabilizes, and periodically enters ProbeRTT to refresh the RTT estimate before returning to ProbeBW.

Stage 1: Implement Startup Behaviour

- The sender estimates the observed bitrate and round-trip time (RTT) over fixed intervals called *phases*, where each phase lasts one RTT.
- When a phase ends, these measurements are used to make a control decision, and the BBR Startup behavior is implemented by doubling the sending rate in each successive phase.
- The bitrate is estimated from the amount of acknowledged data during the phase, while RTT is measured as the time between sending a packet and receiving its corresponding ACK.
- Since the sender initially has no estimates of bitrate or RTT, it starts with a large pacing interval and updates its estimates once the first acknowledgments are received, while the flow-control window still limits the amount of data in flight.

Stage 2: Implement Startup -> ProbeBW transition

- In Stage 2, the sender transitions out of the Startup state once the observed bitrate stops increasing significantly for several consecutive phases.
- Specifically, Startup ends when the bitrate estimate remains within $1.25\times$ of the previous estimate for three consecutive phases.

- Instead of entering the Drain state as in standard BBR, this implementation moves directly to the **ProbeBW** state, where the sender periodically probes the network for available bandwidth.
- In this state, the algorithm cycles through a sequence of gains ($3/4, 1, 1, 1, 1, 1, 1, 5/4$), where most phases maintain the current rate, one phase probes for higher bandwidth, and another reduces the rate to recover from possible overshooting.
- In the original BBR design, the gain cycle is $(1, 1, 1, 1, 1, 1, 5/4, 3/4)$. In this project, the cycle starts with $3/4$ to roughly simulate a drain phase, since the separate Drain state is not implemented.
- Additionally, the sender looks at measurements from several recent phases to get more reliable estimates of bandwidth and RTT.

Stage 3: Adding ProbeRTT

- In this stage, a **ProbeRTT** phase is added to periodically refresh the RTT estimate by temporarily reducing the sending rate and draining the network queue.
- During this phase, only a small number of packets are sent so that queueing delays disappear and a more accurate RTT measurement can be obtained.
- Once a new RTT sample is collected, the algorithm exits ProbeRTT and returns to the normal **ProbeBW** cycle using the updated estimates.
- When returning to ProbeBW, it starts at first Gain of 1 sub-phase of the 8 sub-phases.
- Additionally, if packet loss occurs during a phase, that observation is ignored to avoid corrupting the bitrate and RTT estimates.

Setup

Topology:

The experiments use a **dumbbell topology** with two clients and two servers connected through two routers. Each host connects to its router via an access link, while the routers are connected by a bottleneck link where congestion occurs. Link bandwidth and latency may vary across experiments; commonly used values are 1 ms / 10 Mb/s for access links and 100 ms / 1 Mb/s for the bottleneck link.

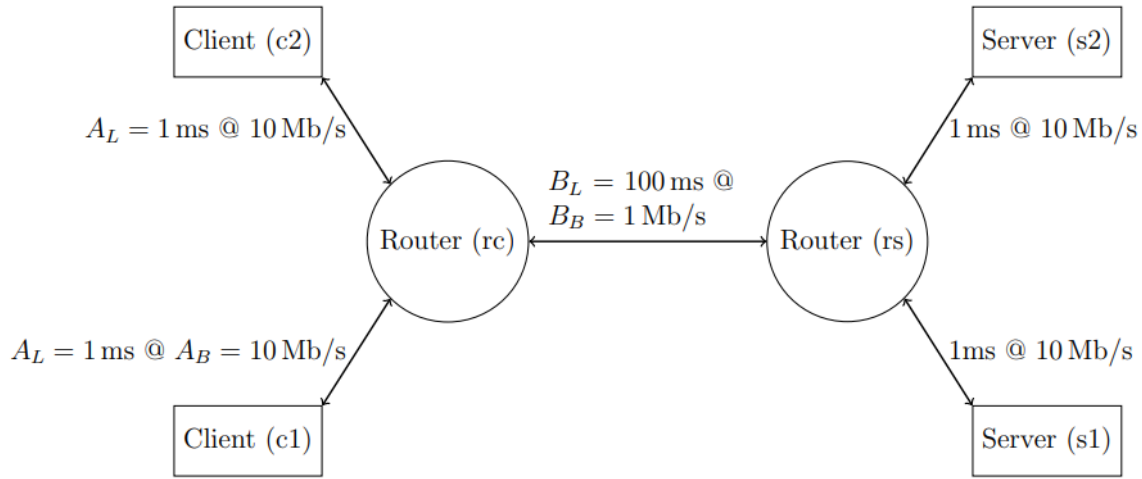


Figure 3: Sample Dumbbell topology used for experiments (Source: Project assignments)

The bottleneck bandwidth (BtBW) for this sample topology is 1 Mb/s, and RTT between a client and server is ($2 \times 102 = 204\text{ms} = 0.204\text{s}$), where 102ms is the one-way delay ($1 + 100 + 1\text{ms}$).

Execution Outputs

1. Sample Phase log

time	seqno	bitrate_bps_fil	rtt_ms_fil	bitrate_bps_inst	rtt_ms_inst	state	gain
...							
0.784003	16384	512000.000000	208.000000	512000.000000	208.000000	STARTUP	2.000000
1.078475	50176	919510.187500	208.000000	919510.187500	294.000000	STARTUP	2.000000
1.364142	83968	948547.375000	208.000000	948547.375000	285.000000	STARTUP	2.000000
1.656281	117760	925808.250000	208.000000	925808.250000	292.000000	STARTUP	2.000000
1.934638	151552	972431.625000	208.000000	972431.625000	278.000000	PROBE_BW	0.750000
2.223829	185344	972431.625000	208.000000	935418.687500	289.000000	PROBE_BW	1.000000
2.504960	219136	972431.625000	278.000000	962049.812500	281.000000	PROBE_BW	1.000000
2.796015	252928	972431.625000	278.000000	928989.687500	291.000000	PROBE_BW	1.000000
3.077359	286720	972431.625000	278.000000	958638.312500	282.000000	PROBE_BW	1.000000
3.400196	320512	972431.625000	278.000000	839552.812500	322.000000	PROBE_BW	1.000000
3.693001	354304	962049.812500	281.000000	925808.250000	292.000000	PROBE_BW	1.000000
4.265371	388096	962049.812500	281.000000	471790.562500	573.000000	PROBE_BW	1.250000
4.540768	421888	983040.000000	275.000000	983040.000000	275.000000	PROBE_BW	0.750000
4.826157	455680	983040.000000	275.000000	951887.312500	284.000000	PROBE_BW	1.000000

The phase log shows the evolution of bandwidth and RTT estimates over time. During the Startup phase, the sending rate increases rapidly with gain 2 until the bandwidth estimate stabilizes near the bottleneck capacity. The

algorithm then transitions to ProbeBW, where it cycles through different gains to probe for available bandwidth and drain queues. The filtered estimates converge to approximately 970 kbps bandwidth and ~208 ms RTT, which are close to the true values.

Column descriptions:

- **time:** time since the start of the connection (seconds)
- **seqno:** sequence number at the start of the phase
- **bitrate_bps_fil:** filtered bandwidth estimate (BtlBw) used by the algorithm: estimated BtlBw
- **rtt_ms_fil:** RTT estimate (RtProp) used by the algorithm: estimated RTT
- **bitrate_bps_inst:** instantaneous bandwidth measured during the current phase
- **rtt_ms_inst:** instantaneous RTT measured during the current phase
- **state:** current BBR state (e.g., STARTUP or PROBE_BW)
- **gain:** pacing gain applied to compute the sending rate

Note: The filtered estimates sometimes remain unchanged across several rows. This happens because the implementation computes BtlBw as the maximum of the last three phase bandwidth observations and RTT as the minimum of the last three phase RTT observations, which helps produce more stable and robust estimates.

2. Time Sequence diagram (first 5 seconds)

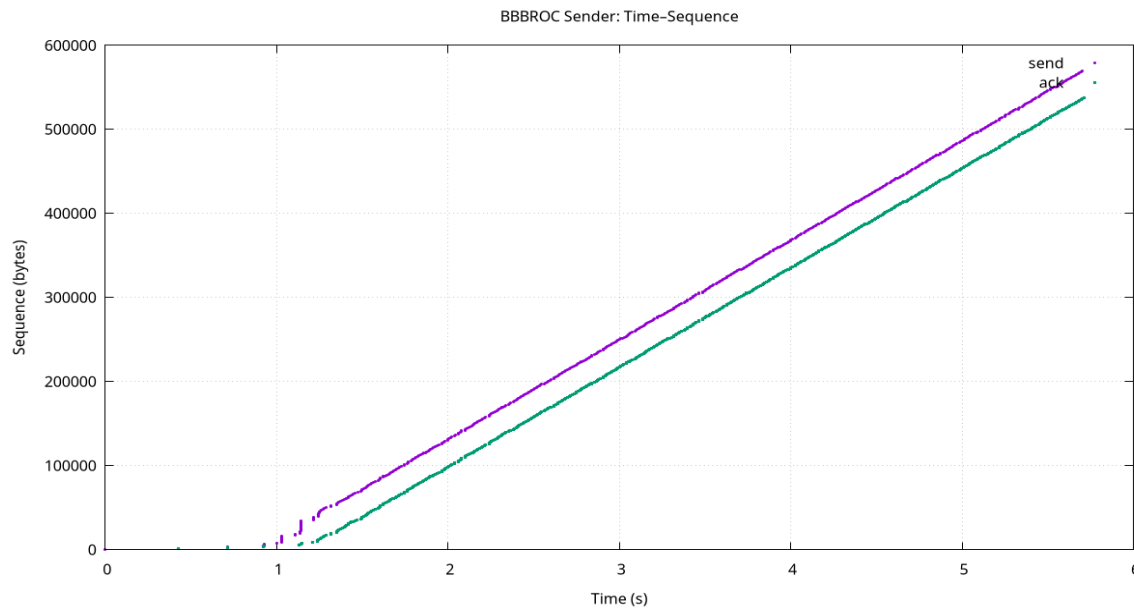


Figure 4: BBR time-sequence plot showing STARTUP ramp-up followed by transition to steady PROBE_BW, where send and ACK rates stabilize into parallel linear growth.

```

cs551@fedora:~/Documents/manasa-CN/test-bbbroc-2025-10-21 $ cat bbbroc.txt
#fsdb -F t time seqno bitrate_bps_fil rtt_ms_fil bitrate_bps_inst rtt_ms_inst state gain
0.424540 0 19366.429688 423.000000 19366.429688 423.000000 STARTUP 2.000000
0.923743 2048 39009.523438 210.000000 39009.523438 210.000000 STARTUP 2.000000
1.132988 4096 39766.992188 206.000000 39766.992188 206.000000 STARTUP 2.000000
1.351682 19456 540981.125000 206.000000 540981.125000 212.000000 STARTUP 2.000000
1.623269 53248 993882.375000 206.000000 993882.375000 272.000000 STARTUP 2.000000
1.907337 87040 951887.312500 206.000000 951887.312500 284.000000 STARTUP 2.000000
2.193928 120832 945230.750000 206.000000 945230.750000 286.000000 STARTUP 2.000000
2.481145 154624 993882.375000 206.000000 938666.687500 288.000000 PROBE_BW 0.750000
2.768527 188416 993882.375000 212.000000 941937.312500 287.000000 PROBE_BW 1.000000
3.050484 222208 993882.375000 272.000000 958638.312500 282.000000 PROBE_BW 1.000000

```

- In the beginning (STARTUP phase), BBR is aggressively probing for bandwidth, which shows up as a steadily increasing slope in the send curve and rising bitrate values in the logs.
- Around ~2.1–2.4 seconds, the bitrate stops increasing, meaning BBR has likely found the bottleneck bandwidth. Right after that, the log switches from STARTUP to PROBE_BW and the pacing gain drops, signaling a transition to steady-state operation.
- In the graph, this corresponds to the send and ack lines becoming straight and parallel, indicating a stable sending rate with controlled queueing.
- So the key transition is visible as “increasing slope → constant slope”, backed by the log’s state change and flattened bitrate.

3. Phase log, showing ProbeRTT effect

- In one of my runs, the RTT estimates (rtt_ms_fil) dropped **down to 209ms from 284ms** after the first ProbeRTT drain, and **from 283ms to 212ms** after the second ProbeRTT drain in the same run.

```

15.671034 1742848 951887.312500 284.000000 948547.375000 285.000000 PROBE_BW 1.250000
16.168807 1792000 951887.312500 209.000000 587942.562500 209.000000 PROBE_BW 1.000000
...
29.759566 3380224 955250.875000 283.000000 951887.312500 284.000000 PROBE_BW 0.750000
30.259164 3430400 955250.875000 212.000000 618264.125000 212.000000 PROBE_BW 1.000000

```

4. Non-adaptive Cross Traffic

- First, I ran a baseline experiment without any competing traffic to verify that the sender correctly estimates the bottleneck bandwidth and operates at full link capacity.
- I then introduced non-responsive UDP cross traffic of 50kb/s during an ongoing transfer, which shares the same bottleneck, to observe how the protocol reacts when available bandwidth is reduced.
(bottleneck bandwidth in this setup was 100 Kb/s)

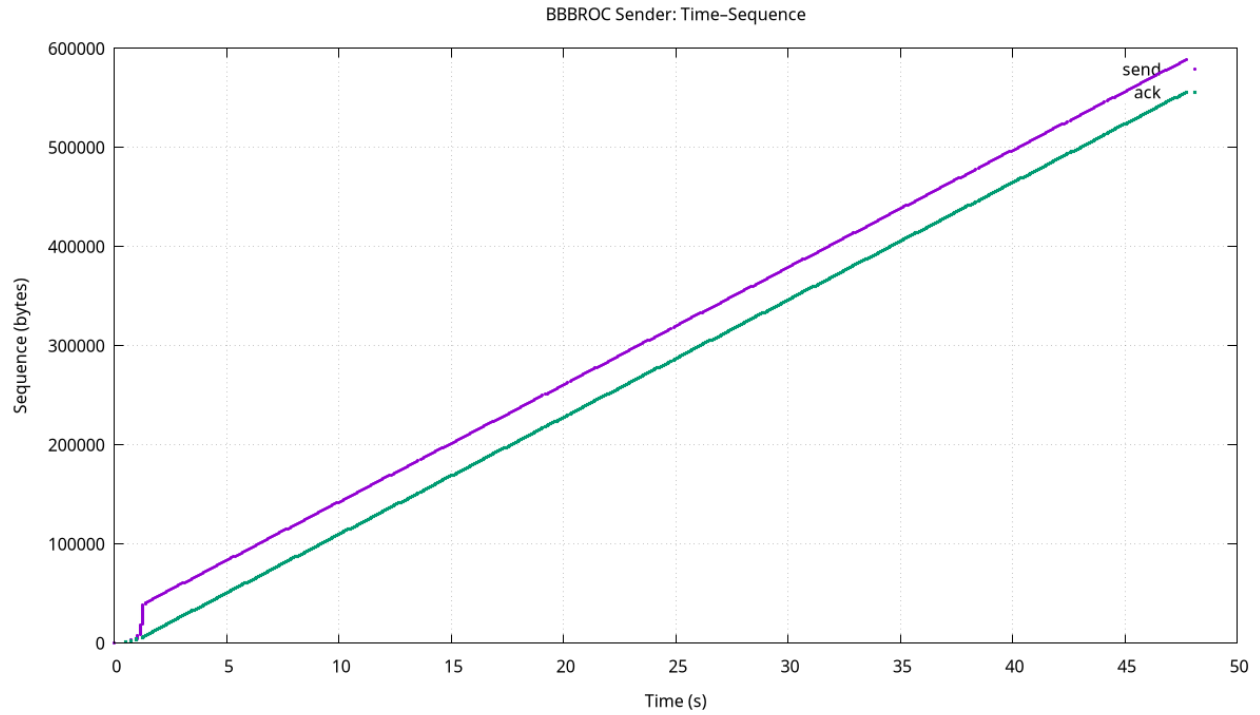


Figure 5: Baseline time-sequence plot showing steady BBR behavior with send and ACK curves progressing linearly and in parallel, indicating stable bandwidth utilization without competing traffic.

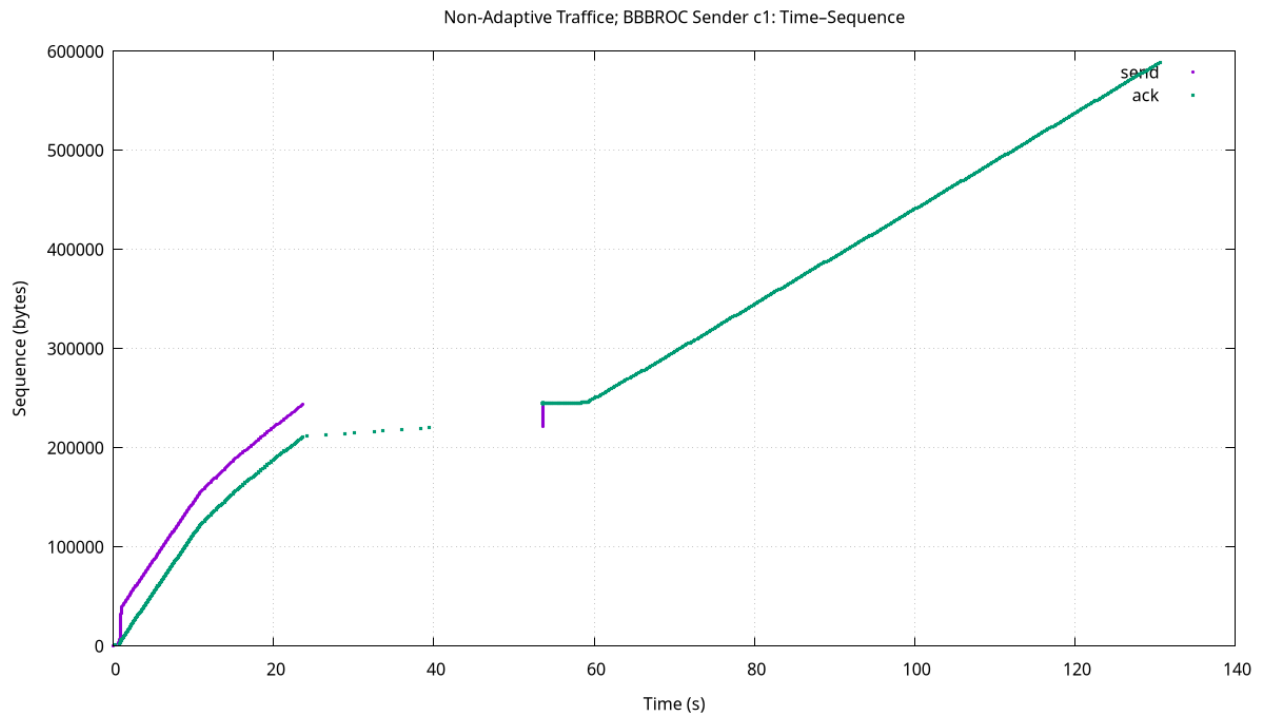


Figure 6: Time-sequence plot showing BBR under non-adaptive UDP cross traffic

```

cs551@fedora:~/Documents/manasa-CN/test-bbbroc-2025-10-21 $ cat b-st3b.txt
#fsdb -F t time seqno bitrate_bps_fil rtt_ms_fil bitrate_bps_inst rtt_ms_inst state gain
0.215588      0      38102.324219    215.000000    38102.324219    215.000000    STARTUP 2.000000
0.935723      4096    113253.453125    215.000000    113253.453125    217.000000    STARTUP 2.000000
1.905117     16384   101448.914062    215.000000    101448.914062    969.000000    STARTUP 2.000000
4.761532     50176   94655.460938    215.000000    94655.460938    2856.000000    STARTUP 2.000000
7.617974     83968   113253.453125    215.000000    94622.328125    2857.000000    PROBE_BW 0.750000
10.491813    117760  113253.453125    215.000000    94062.632812    2874.000000    PROBE_BW 1.000000
14.557369    151552  113253.453125    217.000000    66503.320312    4065.000000    PROBE_BW 1.000000
19.469237    185344  101448.914062    969.000000    55035.832031    4912.000000    PROBE_BW 1.000000

```

Figure 7: Phase log showing BBR under non-adaptive UDP cross traffic

- The plot in Figure 6 and the phase log in Figure 7 show how BBR behaves when non-adaptive UDP cross traffic is introduced.
- At the start, the sender ramps up normally (STARTUP). Around ~14s, observed bitrate starts reducing, and around ~20–40 seconds, you see the ACK curve flatten; this is when the UDP cross traffic occupies part of the bottleneck, reducing available bandwidth and causing BBR's delivery rate to drop. There's a noticeable pause and gap where acknowledgments lag or stall, indicating queue buildup and reduced throughput.
- After this disruption, the flow resumes with a lower, steady slope, showing that BBR has adjusted to the reduced bandwidth and is now operating at a new equilibrium under contention.
- Overall, the key takeaway is that the non-responsive UDP traffic forces BBR to back off, visible as the transition from fast growth → stall → slower linear progression.

5. Adaptive Cross Traffic

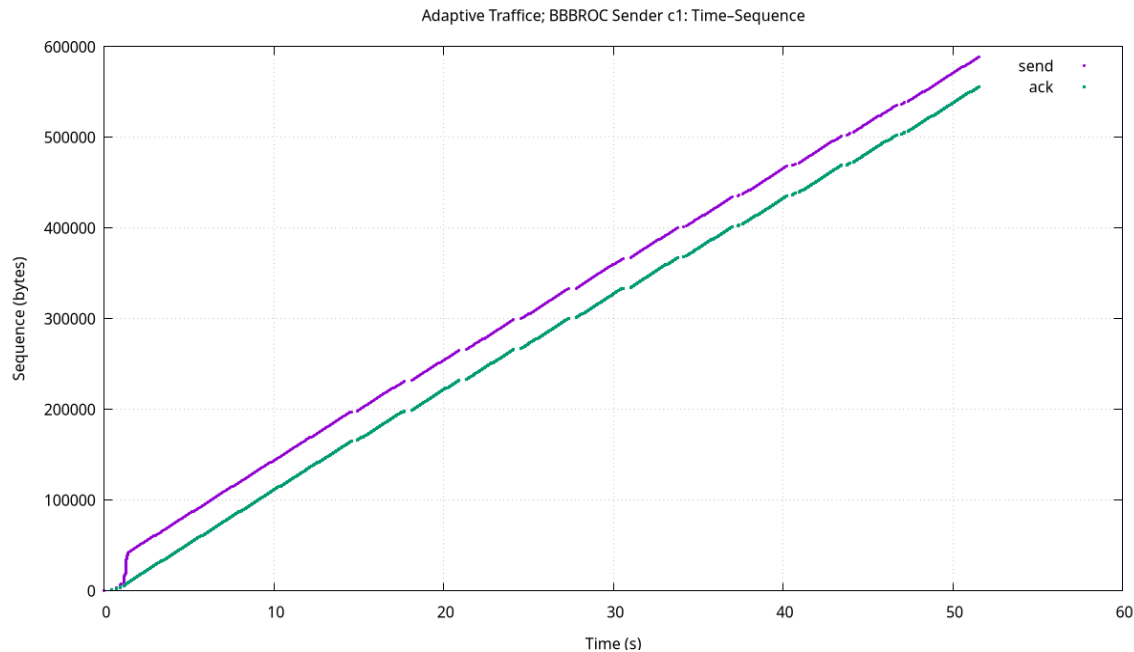


Figure 8: Time-sequence plot showing BBR with adaptive (TCP) cross traffic, where send and ACK curves remain linear and parallel, indicating stable and fair bandwidth sharing despite minor fluctuations.

- I repeated the experiment with adaptive cross traffic by introducing a TCP flow during an ongoing transfer, where the competing traffic reacts to congestion instead of sending at a fixed rate.
- This was done to observe how BBBROC shares bandwidth with another flow that also responds to network congestion.
- The plot in Figure 8 shows BBR operating in the presence of adaptive (TCP) cross traffic.
- After the initial ramp-up, both the send and ACK curves quickly settle into smooth, linear, and closely parallel lines, indicating stable throughput without major stalls.
- Unlike the UDP case, there is no prolonged flattening or disruption in the ACK curve, because the competing TCP flow backs off in response to congestion instead of overwhelming the link.
- This demonstrates BBR's ability to coexist **fairly** with other congestion-controlled flows.
- Minor discontinuities are visible due to ACK clustering, pacing cycles, and measurement granularity, but they do not affect the overall trend.

6. Sample Test Setups (Mininet)

```

css51@fedora:~/Documents/manasa-CN/test-bbbroc-2025-10-21 — sudo python3 dumbbell_topology.py -c
css51@fedora:~/Documents/manasa-CN/test-bbbroc-2025-10-21 $ sudo python3 dumbbell_topology.py -c
[sudo] password for css51:
*** Creating network
*** Adding hosts:
c1 c2 rc rs s1 s2
*** Adding switches:

*** Adding links:
(10.00Mbit 1ms delay 0.00000% loss) (10.00Mbit 1ms delay 0.00000% loss) (c1, rc) (10.00Mbit 100ms delay 0.00000% loss) (10.00Mbit 100ms delay 0.00000% loss) (rc, rs) (10.00Mbit 1ms delay 0.00000% loss) (10.00Mbit 1ms delay 0.00000% loss) (s1, rs) (10.00Mbit 1ms delay 0.00000% loss)
*** Configuring hosts
c1 c2 rc rs s1 s2
*** Starting controller

*** Starting 0 switches

*** Starting mininet CLI
*** Starting CLI:
mininet> xterm c1 s1
mininet>
  
```

"Node: c1"

```

t 33792
bbbrc_send: ...but window limited, snd size: 0.
bbbrc_send: checking for acks
bbbrc_send: want to send, snd_a 5209088 vs size 5242887
bbbrc_send: cb->snd_bdp 25683, max_bitrate 983040, min_rtt 209, cb->snd_inflight 33792
t 33792
bbbrc_send: ...but window limited, snd size: 0.
bbbrc_send: checking for acks
bbbrc_send: want to send, snd_a 5209088 vs size 5242887
bbbrc_send: cb->snd_bdp 25683, max_bitrate 983040, min_rtt 209, cb->snd_inflight 33792
t 33792
bbbrc_send: ...but window limited, snd size: 0.
bbbrc_send: checking for acks
bbbrc_recv_ack: recvd ack at 1773726657,304063
bbbrc_recv_ack: got ack 5210112, snd_a now 5210112, cb->snd_record_seqno is 5239808
bbbrc_send: want to send, snd_a 5210112 vs size 5242887
bbbrc_send: cb->snd_bdp 25683, max_bitrate 983040, min_rtt 209, cb->snd_inflight 32768
t 32768
bbbrc_send_new_segment: at 1773726657,304094
bbbrc_send_new_segment: will send FIN
bbbrc_send_specific_segment: send new @ 5242880 for 7, pacing_delay is 8, cur state: 1, snd_estimated_bitrate: 983040,000000
[root@fedora test-bbbroc-2025-10-21]#
          
```

"Node: s1"

```

[root@fedora test-bbbroc-2025-10-21]# ./bbbrc_recvfile s1.txt
[root@fedora test-bbbroc-2025-10-21]#
  
```

Figure X: Baseline setup with a single BBBROC flow (one sender and one receiver), fully utilizing the bottleneck bandwidth.

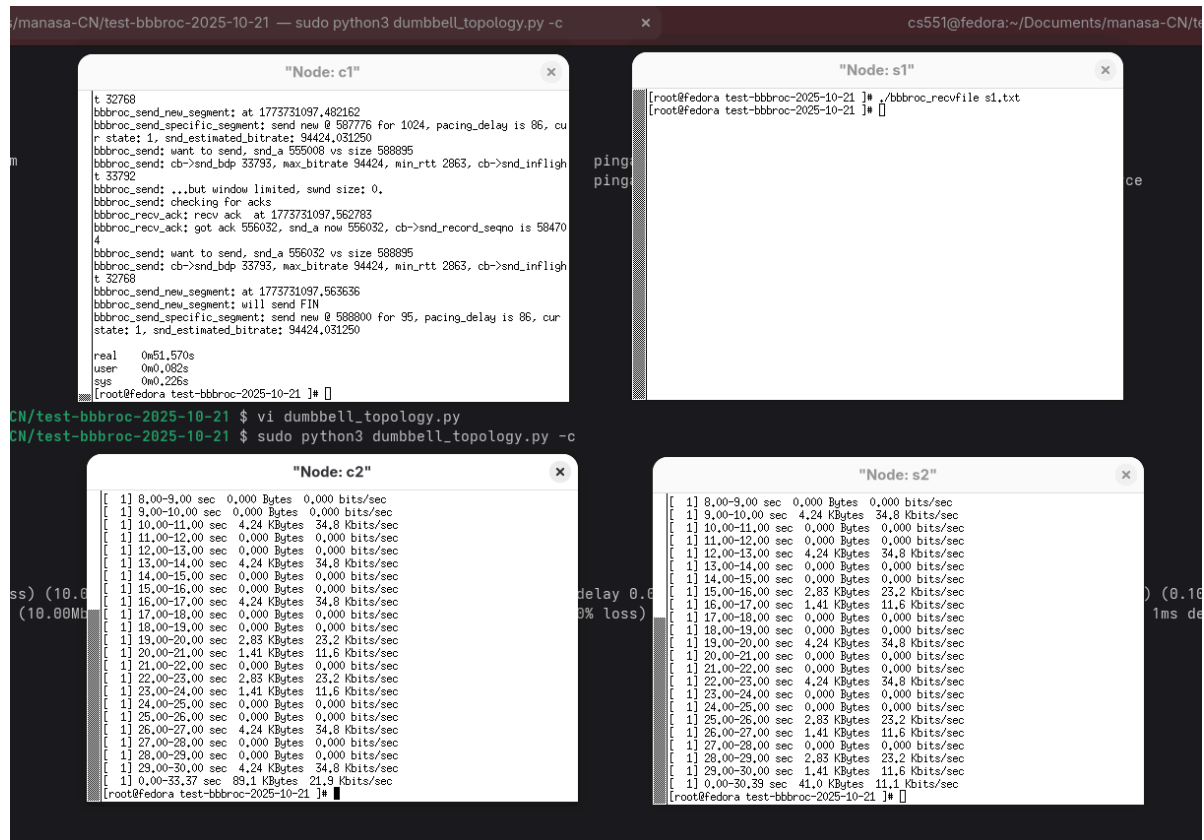


Figure X: Experimental setup for adaptive cross traffic, showing BBROOC sender/receiver alongside competing TCP flows sharing the bottleneck link.

7. Additional Experiments in the Appendix

Several additional experiments were conducted, including TCP solo, TCP vs TCP, BBROOC vs BBROOC, BBROOC under varying RTTs, and ProbeRTT behavior with changing RTT conditions. These are included in detail in the Appendix.

Appendix Contents

The Appendix also contains supporting materials such as phase logs, setup screenshots, and additional time-sequence graphs for deeper analysis, especially BBR **fairness**